VHAL Foundations — Detailed Study Guide

This document expands on each of the 18 foundational VHAL questions with deeper explanations, internal mechanics, and concrete examples. It's meant as a reference you can revisit as you start reading actual AOSP source code.

---

1. What is VHAL, and why does it exist?

Every Android HAL exists for the same fundamental reason: **separation between Google's framework code and a vendor's hardware-specific implementation**. This is part of what's historically called "Project Treble" — the idea that the Android framework (system partition) and vendor-specific code (vendor partition) should be cleanly separated, communicating only through stable, versioned interfaces (HIDL/AIDL).

For automotive, this separation is *especially* important because vehicle hardware is wildly non-standard across manufacturers:

- A Toyota's HVAC system might communicate over a CAN bus with one set of message IDs and byte layouts.
- A Ford's HVAC system uses a completely different CAN topology, possibly with a CAN-FD or even an Ethernet/SOME-IP backbone.
- Some vehicles route certain signals through a body control module (BCM), others through a dedicated climate ECU.

Without VHAL, every OEM would need to modify the Android framework itself to talk to their specific hardware — meaning Google couldn't ship framework updates without breaking every OEM's customizations. VHAL solves this by defining a **property-based contract**: as long as an OEM's VHAL implementation can answer "what is the value of property X" and "set property X to value Y," the rest of Android (framework, apps) never needs to know *how* that happens underneath.

Concrete walkthrough: A user taps "+" on a climate control app to raise the cabin temperature.

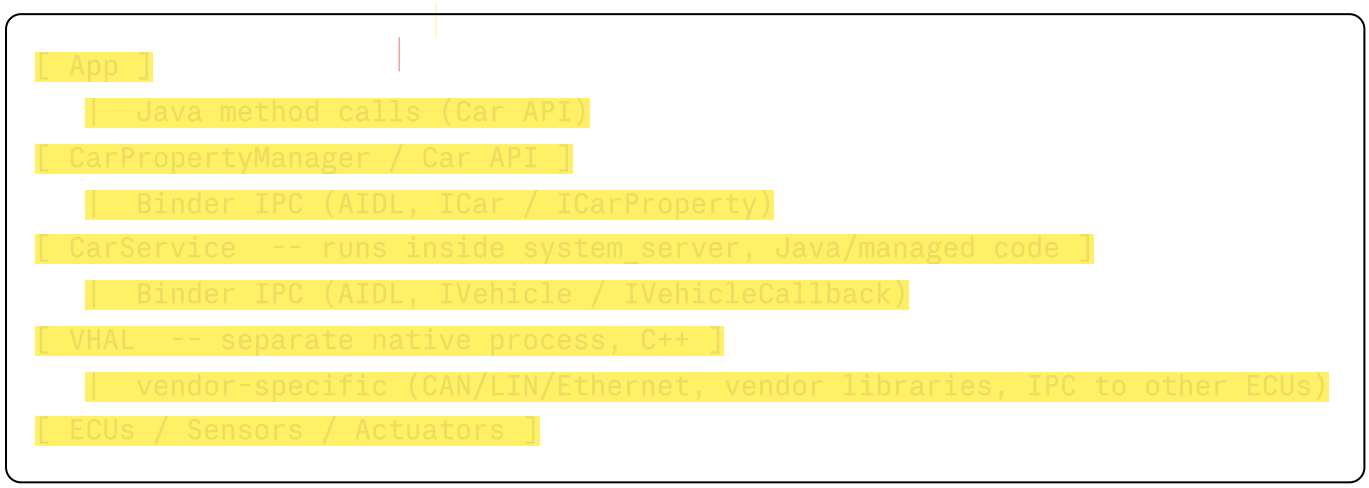
1. The app calls `CarPropertyManager.setFloatProperty(HVAC_TEMPERATURE_SET, areaId, 23.0f)`.
2. CarService checks the app has the right permission, then calls VHAL's `setValues()` via Binder.
3. The OEM's VHAL implementation receives this and translates "23.0f for driver seat zone" into whatever the HVAC ECU expects — e.g., constructing a specific CAN frame with ID `0x3A1` and encoding 23.0°C as a particular byte value.
4. That CAN frame is sent out over the vehicle bus, the HVAC ECU receives it, and physically adjusts a blend-door actuator.

Everything above VHAL (the app, CarService, the Car API) is **identical across every car** running AAOS. Only the VHAL implementation differs per OEM.

---

2. Where does VHAL fit in the overall AAOS architecture?

The layered stack, with what crosses each boundary:



Two distinct IPC boundaries exist here, and they matter for different reasons:

- **App ↔ CarService**: This is normal Android app-to-system-service Binder IPC, the same mechanism any app uses to talk to `ActivityManagerService`, `LocationManagerService`, etc.
- **CarService ↔ VHAL**: This is the *HAL boundary*. It crosses from managed Java code (`system_server`) into a separate native process. This is the boundary that's part of the "vendor partition" split — an OEM can update their VHAL binary independently of the Android framework, as long as it still speaks the same AIDL `IVehicle` interface version.

The reason VHAL is a **separate process** (not just a native library loaded into `system_server`) is isolation: a crash, hang, or memory corruption in vendor-written VHAL code cannot directly crash `system_server` (which would be catastrophic — it hosts dozens of critical system services). If VHAL crashes, CarService detects the broken Binder connection, and (depending on implementation) can attempt to reconnect or restart it, while the rest of the Android framework keeps running.

---

3. What is a "Vehicle Property"?

A property is identified by a 32-bit integer ID, and that ID isn't arbitrary — it's a **bitfield** encoding metadata about the property. The high-order bits encode three things: the property *group* (who defined it), the *area type* (what kind of zone it applies to), and the *data type* (what kind of value it carries). The low-order bits are a unique "base ID" within those categories.

For example, `PERF_VEHICLE_SPEED` has the value `0x11600207`, which breaks down as:

Bits	Meaning	Value
Group	SYSTEM (defined by AOSP)	`0x10000000`
Area	GLOBAL (applies to whole vehicle)	`0x01000000`
Type	FLOAT	`0x00600000`
Base ID	unique identifier	`0x0207`

You'll see this same pattern across all properties. A few common ones to recognize:

- `GEAR_SELECTION` — INT32, GLOBAL — current gear (Park/Reverse/Neutral/Drive)
- `PERF_VEHICLE_SPEED` — FLOAT, GLOBAL — current speed in m/s
- `HVAC_TEMPERATURE_SET` — FLOAT, SEAT — target temperature, per-seat
- `DOOR_LOCK` — BOOLEAN, DOOR — lock state, per-door
- `FUEL_LEVEL` — FLOAT, GLOBAL — remaining fuel
- `NIGHT_MODE` — BOOLEAN, GLOBAL — whether ambient light sensor indicates night

Once you internalize this bitfield structure, property IDs stop looking like random hex numbers and start being self-describing — you can often guess a property's type and scope just by looking at the ID.

---

4. What is `VehiclePropValue`?

This is the "envelope" that carries an actual data point. Its fields (AIDL definition, simplified):

```
parcelable VehiclePropValue {
    long timestamp;        // nanoseconds since boot, when this value was
sampled
    int areaId;            // which zone this value applies to (0 = GLOBAL)
    int prop;              // the property ID (e.g., HVAC_TEMPERATURE_SET)
    VehiclePropertyStatus status;  // AVAILABLE, UNAVAILABLE, ERROR
    RawPropValues value;  // the actual data, in one of several typed arrays
}
```

`RawPropValues` itself contains separate arrays for each possible type — `int32Values[]`, `int64Values[]`, `floatValues[]`, `byteValues[]`, and `stringValue`. Only the array(s) matching the property's declared type are populated; the rest are empty. This is a common

pattern in IPC-friendly type systems (rather than using a generic "Object" type, which doesn't serialize cleanly across process boundaries).

Concrete example — reading the driver's HVAC temperature setting might return:

```
VehiclePropValue {
    timestamp: 123456789000,
    areaId: 0x0001,             // ROW_1_LEFT (driver seat)
    prop: HVAC_TEMPERATURE_SET, // FLOAT type property
    status: AVAILABLE,
    value: { floatValues: [22.5] }
}
```

The `status` field matters a lot in practice — a property can be temporarily `UNAVAILABLE` (e.g., a sensor that hasn't initialized yet) or `ERROR` (e.g., a CAN bus timeout), and CarService/apps need to handle these states gracefully rather than assuming every read succeeds with valid data.

---

5. What is `VehiclePropConfig`?

While `VehiclePropValue` is *data*, `VehiclePropConfig` is *metadata describing the property's capabilities*. VHAL must return one of these for every property it supports, via `getPropertyConfigs()` / `getAllPropertyConfigs()`. Key fields:

```
parcelable VehiclePropConfig {
    int prop;
    VehiclePropertyAccess access;       // READ / WRITE / READ_WRITE
(default)
    VehiclePropertyChangeMode changeMode; // STATIC / ON_CHANGE / CONTINUOUS
    VehicleAreaConfig[] areaConfigs;     // per-zone configs (min/max, access
overrides)
    int[] configArray;                  // extra property-specific config
data
    String configString;
    float minSampleRate;  // Hz, for CONTINUOUS properties
    float maxSampleRate;  // Hz, for CONTINUOUS properties
}
```

`VehicleAreaConfig` is where things get interesting for multi-zone properties:

```
parcelable VehicleAreaConfig {
    int areaId;
    int minInt32Value; int maxInt32Value;
    int minInt64Value; int maxInt64Value;
    float minFloatValue; float maxFloatValue;
    VehiclePropertyAccess access;  // can override the property-level default
}
```

Example: `HVAC_TEMPERATURE_SET` might have `areaConfigs` like:

- `areaId = ROW_1_LEFT (driver)`, min=16.0, max=32.0
- `areaId = ROW_1_RIGHT (passenger)`, min=16.0, max=32.0
- `areaId = ROW_2_LEFT | ROW_2_RIGHT (rear, shared zone)`, min=18.0, max=28.0

CarService reads these configs once at startup and uses them to validate every subsequent `set()` call from an app *before* even bothering VHAL — e.g., rejecting a request to set the driver's temperature to 50°C because it's outside `[16.0, 32.0]`.

---

6. What are property groups (SYSTEM, VENDOR, BACKPORTED)?

This is the top bit-group of a property ID, and it answers "who defined this property and what guarantees come with it?"

- **`SYSTEM`** (**`0x10000000`**): Properties defined in AOSP's `VehicleProperty.aidl`. Every AAOS device, regardless of OEM, is expected to understand these IDs the same way — `GEAR_SELECTION` always means the same thing on every car. Apps written against these properties are portable across vehicles (assuming the vehicle actually supports/reports that property).
- **`VENDOR`** (**`0x20000000`**): Properties an OEM invents for features that don't exist in the standard set. For example, a manufacturer might define `VENDOR_AMBIENT_LIGHT_COLOR` or `VENDOR_MASSAGE_SEAT_INTENSITY`. These IDs are entirely OEM-defined (with the base-ID portion chosen by the OEM), and apps need OEM-specific knowledge (and usually OEM-granted custom permissions) to use them.
- **`BACKPORTED`** (**`0x30000000`**): This solves a versioning problem. Suppose Android 14 introduces a new SYSTEM property, but a particular vehicle's VHAL was built against the Android 13 `IVehicle` interface and can't be rebuilt immediately. The OEM can implement the *new* property's behavior under a `BACKPORTED`-prefixed ID using their existing vendor extension mechanism, and CarService (updated to a newer framework version) knows to map/recognize it as the equivalent of the new SYSTEM property. This lets newer framework features work on slightly older VHAL implementations without a full VHAL rebuild, at the cost of the OEM doing a bit of extra mapping work.

---

7. What are the access modes for a property?

Defined by `VehiclePropertyAccess`:

```
enum VehiclePropertyAccess {
    NONE = 0,
    READ = 1,
    WRITE = 2,
    READ_WRITE = 3,
}
```

This is set at the property level in `VehiclePropConfig.access`, and can be **overridden per-area** in `VehicleAreaConfig.access`. Why would you need per-area overrides? Consider a hypothetical seat-occupancy-based property: the driver's seat sensor might be read-write (for calibration), while rear seat sensors are read-only to regular apps for privacy/safety reasons.

Typical examples:

- `PERF_VEHICLE_SPEED` → `READ` (an app obviously cannot "set" the car's actual speed)
- `HVAC_TEMPERATURE_SET` → `READ_WRITE` (app can read current target and change it)
- `INFO_VIN` (vehicle identification number) → `READ`
- `DOOR_LOCK` → `READ_WRITE` (read current lock state, or command lock/unlock)

CarService enforces this: if an app calls `setFloatProperty()` on a `READ`-only property, CarService throws an exception *before* even contacting VHAL.

---

8. What are change modes, and why do they matter?

`VehiclePropertyChangeMode` governs how/when VHAL pushes updates for a property to subscribers:

```
enum VehiclePropertyChangeMode {
    STATIC = 0,
    ON_CHANGE = 1,
    CONTINUOUS = 2,
}
```

- `STATIC`: The value is fixed for the lifetime of the system — VHAL never sends update events for it. Example: `INFO_MAKE`, `INFO_MODEL`, `INFO_VIN`. CarService typically caches these after the first read.

- `ON_CHANGE`: VHAL sends an `onPropertyEvent` callback *only when the underlying value actually changes* — no periodic polling. Example: `DOOR_LOCK` (you don't need 10 updates/sec telling you the door is still locked — you only care the instant it changes), `NIGHT_MODE`, `GEAR_SELECTION`.
- `CONTINUOUS`: VHAL streams updates at a sample rate, even if the value hasn't meaningfully changed, because the *rate of change itself* is useful information. Example: `PERF_VEHICLE_SPEED`, `WHEEL_TICK`, accelerometer-style data. The config's `minSampleRate`/`maxSampleRate` (in Hz) bound what rate a subscriber can request — e.g., min=1Hz, max=10Hz means a subscriber can ask for anywhere between 1 and 10 updates per second.

The practical impact: if you write code that subscribes to an `ON_CHANGE` property expecting periodic ticks, you'll be disappointed — you'll only get callbacks on actual transitions. Conversely, subscribing to a `CONTINUOUS` property and assuming "no callback = no change" is wrong — you'll get callbacks at the sample rate regardless.

---

9. What is an "Area" or `AreaId`?

Many vehicle properties aren't single global values — they're replicated per physical zone. The `AreaId` tells you *which zone* a particular `VehiclePropValue` refers to, and it's a **bitmask**, not a simple enum index. This matters because a single area-config entry can apply to *multiple* zones simultaneously by OR-ing their bits together.

`VehicleAreaSeat` bit flags (illustrative):

```
ROW_1_LEFT   = 0x0001
ROW_1_CENTER = 0x0002
ROW_1_RIGHT  = 0x0004
ROW_2_LEFT   = 0x0010
ROW_2_CENTER = 0x0020
ROW_2_RIGHT  = 0x0040
ROW_3_LEFT   = 0x0100
ROW_3_CENTER = 0x0200
ROW_3_RIGHT  = 0x0400
```

Why bitmasks? Suppose a vehicle's rear climate zone is controlled by a single shared knob — both rear-left and rear-right passengers get the same temperature. Rather than VHAL maintaining two separate config entries with duplicated min/max ranges, it can declare *one* `VehicleAreaConfig` with `areaId = ROW_2_LEFT | ROW_2_RIGHT` (`0x0010 | 0x0040 = 0x0050`). When CarService gets a `VehiclePropValue` with `areaId = 0x0050`, it knows this single value applies to both rear seats.

When an *app* calls `getFloatProperty(HVAC_TEMPERATURE_SET, areaId)`, it typically passes the specific single-seat bit it cares about (e.g., `ROW_1_LEFT = 0x0001`), and CarService

figures out which config entry (possibly a combined one) covers that bit.

`GLOBAL` area (`areaId = 0`) is the special case for properties that aren't zoned at all — there's exactly one value for the whole vehicle, and exactly one `VehicleAreaConfig` entry with `areaId = 0`.

Other area types follow the same bitmask pattern: `VehicleAreaDoor` (ROW_1_LEFT, ROW_1_RIGHT, ROW_2_LEFT, ROW_2_RIGHT, HOOD, REAR), `VehicleAreaWindow` (FRONT_WINDSHIELD, REAR_WINDSHIELD, ROW_1_LEFT, ROW_1_RIGHT, ROOF_TOP_1, etc.), `VehicleAreaMirror`, `VehicleAreaWheel`.

---

10. How does VHAL actually communicate with CarService?

Through the AIDL interface `IVehicle`, defined under `hardware/interfaces/automotive/vehicle/aidl/android/hardware/automotive/vehicle/IVehicle.aidl`. The current (AIDL-based, Android 13+) interface is **asynchronous**: most calls don't return data directly — instead, you pass a callback object, and results arrive later via that callback. This is a deliberate design choice for automotive: a `set()` call that goes out over a CAN bus might take noticeable time to confirm, and you don't want to block a Binder thread waiting.

Core methods (simplified):

```
interface IVehicle {
    VehiclePropConfig[] getAllPropertyConfigs();

    void getValues(in GetValueRequests requests, in IVehicleCallback
callback);
    void setValues(in SetValueRequests requests, in IVehicleCallback
callback);

    void subscribe(in IVehicleCallback callback, in SubscribeOptions[]
options, int maxSharedMemoryFileCount);
    void unsubscribe(in IVehicleCallback callback, in int[] propIds);
}
```

- `getAllPropertyConfigs()` — called once at startup; CarService uses this to build its internal map of "what properties exist, what types/ranges/access modes they have."
- `getValues(requests, callback)` — request to read current value(s) of one or more properties; results come back via `callback.onGetValues(...)`.
- `setValues(requests, callback)` — request to write value(s); results (success/failure per property) come back via `callback.onSetValues(...)`.
- `subscribe(callback, options, ...)` — register interest in ongoing updates for `ON_CHANGE`/`CONTINUOUS` properties, optionally specifying a sample rate per property in

`SubscribeOptions`.

- `unsubscribe(callback, propIds)` — stop receiving updates.

Historical note: Android 10/11 used a HIDL interface (`android.hardware.automotive.vehicle@2.0::IVehicle`) with a more synchronous-feeling `get()`/`set()`/`subscribe()` API. The AIDL version (current standard since Android 13) restructured things to be explicitly async with request/response objects — useful to recognize if you're reading older code or AOSP branches.

---

11. What is `IVehicleCallback`?

This is the interface VHAL uses to call back *into* CarService — the "reverse direction" of `IVehicle`. CarService implements this interface and hands an instance of it to VHAL when calling `getValues`, `setValues`, or `subscribe`. Its methods (simplified):

```
interface IVehicleCallback {
    void onGetValues(in GetValueResults responses);
    void onSetValues(in SetValueResults responses);
    void onPropertyEvent(in VehiclePropValues propValues, int
sharedMemoryFileCount);
    void onPropertySetError(in VehiclePropErrors errors);
}
```

- `onGetValues` / `onSetValues` — deliver the results of a previously-issued `getValues`/`setValues` call (success with data, or error status per-property).
- `onPropertyEvent` — the key one for subscriptions: whenever a subscribed `ON_CHANGE` property changes, or a `CONTINUOUS` property's sample timer fires, VHAL calls this with a batch of new `VehiclePropValue`s. CarService's implementation of this method is what triggers the chain of dispatching to app-level `CarPropertyEventCallback`s.
- `onPropertySetError` — reports asynchronous failures for set operations that VHAL couldn't complete (e.g., the CAN bus rejected the command).

Understanding this interface is the key to understanding that **VHAL is not passive** — it's not just sitting there waiting to be polled. It actively pushes data into CarService whenever something relevant happens, which is what makes real-time dashboard updates (speed, RPM, etc.) possible.

---

12. What is CarService, and how does it relate to VHAL?

CarService is a system service that runs inside `system_server` (the same process hosting `ActivityManagerService`, `PackageManagerService`, `WindowManagerService`, etc.), under the package `com.android.car` / source path `packages/services/Car/service/`. It's the **single**

point of contact with VHAL — no app, and no other system service, talks to VHAL directly. This centralization lets CarService:

1. **Enforce permissions** — checking that an app calling `setFloatProperty(HVAC_TEMPERATURE_SET, ...)` actually holds `CONTROL_CAR_CLIMATE` before forwarding to VHAL.
2. **Multiplex subscriptions** — if three different apps all subscribe to `PERF_VEHICLE_SPEED`, CarService maintains exactly *one* VHAL subscription (at the highest requested sample rate among the three) and fans the resulting `onPropertyEvent` callbacks out to all three app listeners. VHAL never knows or cares how many apps are interested.
3. **Cache and validate** — for `STATIC` properties, cache the value after first read; for all properties, validate `set()` requests against the `VehiclePropConfig` ranges before bothering VHAL.
4. **Translate types** — internally, CarService wraps the raw AIDL `VehiclePropValue`/`VehiclePropConfig` in its own `HalPropValue`/`HalPropConfig` abstractions (found in `VehicleHal.java` and related classes), partly to insulate the rest of CarService from AIDL-vs-HIDL differences.

CarService then exposes a *separate*, app-facing AIDL interface (things like `ICarProperty`) that `CarPropertyManager` talks to. So there are genuinely two different IPC contracts here: the app-facing Car API contract (stable, documented for app developers) and the `IVehicle` HAL contract (stable for OEMs implementing VHAL) — and CarService's job is to sit in the middle and bridge them.

---

13. What is `CarPropertyManager`, and how would an app use it?

`CarPropertyManager` is the class app developers actually call — found in `android.car.hardware.property.CarPropertyManager`. You obtain it via the `Car` object:

```java
Car car = Car.createCar(context);
CarPropertyManager propertyManager =
    (CarPropertyManager) car.getCarManager(Car.PROPERTY_SERVICE);
```

Synchronous reads (for properties that don't change often, or where you just need a snapshot):

```java
```

```java
int gear = propertyManager.getIntProperty(
    VehiclePropertyIds.GEAR_SELECTION, /* areaId= */ 0);

float driverTemp = propertyManager.getFloatProperty(
    VehiclePropertyIds.HVAC_TEMPERATURE_SET, VehicleAreaSeat.ROW_1_LEFT);
```

Writes:

java

```java
propertyManager.setFloatProperty(
    VehiclePropertyIds.HVAC_TEMPERATURE_SET,
    VehicleAreaSeat.ROW_1_LEFT,
    23.0f);
```

Subscriptions (for `ON_CHANGE`/`CONTINUOUS` properties like speed):

java

```java
CarPropertyManager.CarPropertyEventCallback callback =
    new CarPropertyManager.CarPropertyEventCallback() {
        @Override
        public void onChangeEvent(CarPropertyValue value) {
            float speed = (Float) value.getValue();
            // update UI
        }
        @Override
        public void onErrorEvent(int propId, int areaId) {
            // handle error
        }
    };

propertyManager.registerCallback(
    callback,
    VehiclePropertyIds.PERF_VEHICLE_SPEED,
    CarPropertyManager.SENSOR_RATE_NORMAL);
```

Behind the scenes, `getIntProperty`/`getFloatProperty`/`setFloatProperty` make Binder calls into CarService, which (if needed) calls VHAL's `getValues`/`setValues` and waits for the async callback before returning a result to the app. `registerCallback` causes CarService to (if not already subscribed) call VHAL's `subscribe()`, and from then on, `onPropertyEvent` data flows back up through CarService into your `onChangeEvent`.

---

14. What's the "default"/"reference" VHAL implementation used for?

AOSP ships a complete, working VHAL implementation that requires **no real vehicle hardware at all** — located under `hardware/interfaces/automotive/vehicle/aidl/impl/`. This is what runs when you boot the Android Automotive emulator (the "Automotive" system images you can create in Android Studio's AVD Manager).

How it works:

- Property configurations and **initial values** are defined declaratively in JSON files (e.g., `DefaultProperties.json` and related config files under the `default_config` directory) — each entry specifies the property ID, access mode, change mode, area configs, and a starting value.
- A C++ class (historically `FakeVehicleHardware`, part of the broader `DefaultVehicleHal`) loads these JSON configs at startup and serves `getValues`/`setValues`/`subscribe` purely from an in-memory map — there's no CAN bus, no kernel driver, nothing below it.
- For `CONTINUOUS` properties, it can simulate periodic updates using simple internal logic (e.g., slowly ramping a value) rather than reading from real sensors.

This is *enormously* useful for development:

- You can develop and test an entire app (climate control UI, etc.) against realistic property behavior without any vehicle.
- AOSP/Google provide a **VHAL emulator tool** (a Python script, often referred to as `vhal_emulator.py`, communicating over a forwarded `adb` port) that lets you manually inject arbitrary `VehiclePropValue` updates into the running reference VHAL — e.g., force `GEAR_SELECTION` to `REVERSE` to test whether your app correctly shows a reverse-camera view, without needing to actually put a car in reverse.

There's also a **gRPC-based** variant of the reference implementation, useful for running VHAL as a separate component (even on a separate machine/container) from the Android system image — handy for certain CI/testing setups.

---

15. How would you add a custom (vendor) property?

Walking through this end-to-end (using the AOSP reference implementation as the example target):

1. **Pick a property ID** in the `VENDOR` range (top bits = `0x20000000`), combined with the appropriate area-type and value-type bits, and a base ID that doesn't collide with anything else your VHAL already defines. E.g., `VENDOR_AMBIENT_LIGHT_COLOR = VENDOR | GLOBAL | INT32 | 0x0F00`.

2. **Define its config.** For the reference implementation, you'd add a new entry to the JSON config (alongside the existing `DefaultProperties.json` entries) specifying `prop`, `access` (likely `READ_WRITE`), `changeMode` (likely `ON_CHANGE`), area configs (e.g., `GLOBAL`, with `minInt32Value`/`maxInt32Value` representing an RGB-packed range), and an initial value.
3. **Implement the get/set behavior.** For a real vehicle, this is where your C++ VHAL code would translate `setValues` calls for this property into whatever talks to the actual ambient-lighting ECU (CAN frame, I2C command to a local microcontroller, etc.). For the reference/emulator implementation, the default in-memory map handles this automatically — no extra code needed beyond the JSON entry.
4. **Define a custom permission.** Vendor properties usually need a vendor-defined Android permission (e.g., `com.yourcompany.car.permission.CONTROL_AMBIENT_LIGHT`), declared with `<permission>` in a system manifest, with an appropriate `protectionLevel` (often `signature` or `privileged` for vehicle-control permissions).
5. **Expose it to apps.** Apps declare `<uses-permission android:name="com.yourcompany.car.permission.CONTROL_AMBIENT_LIGHT"/>` and then simply call `CarPropertyManager.setIntProperty(VENDOR_AMBIENT_LIGHT_COLOR, GLOBAL, colorValue)` — using the raw vendor property ID constant (which the OEM would publish in a small SDK/AAR for third-party developers, or keep internal for first-party apps only).

The important takeaway: **adding a vendor property doesn't require touching the AOSP framework or CarService's core logic at all** — it's entirely additive, which is exactly the point of the HAL abstraction.

---

16. Why do permissions matter so much for VHAL properties?

Vehicle data spans a huge range of sensitivity — some of it is harmless (current radio station), some is safety-critical (ability to *set* gear or unlock doors), and some is privacy-sensitive (location-adjacent data, seat occupancy, driver biometrics in some implementations). Android's permission system is the gatekeeper CarService uses to decide whether to even *attempt* a VHAL call on an app's behalf.

Each property is associated (in CarService's internal configuration, e.g., `PropertyHalServiceConfigs`) with one or more Android permission strings. Examples:

- `android.car.permission.CAR_INFO` — static vehicle info (make, model, VIN)
- `android.car.permission.CAR_SPEED` — `PERF_VEHICLE_SPEED`
- `android.car.permission.CAR_ENERGY` — fuel/battery level
- `android.car.permission.CONTROL_CAR_CLIMATE` — HVAC set-points
- `android.car.permission.CONTROL_CAR_DOORS` — door lock/unlock
- `android.car.permission.CONTROL_CAR_WINDOWS` — window position

When an app calls `CarPropertyManager.getXProperty()` or `setXProperty()`, CarService calls something like `context.checkCallingOrSelfPermission(requiredPermission)` *before* constructing the AIDL request to VHAL. If the check fails, the app gets a `SecurityException` immediately — VHAL is never even contacted. This means permission enforcement happens entirely in the trusted CarService layer, which is important: you don't want permission logic duplicated (and potentially inconsistently implemented) across every OEM's VHAL binary.

Some permissions are also tiered by protection level — many vehicle-control permissions (doors, windows, climate writes) are `signature|privileged`, meaning only apps signed with the platform key or pre-installed as privileged system apps can hold them, not arbitrary Play Store apps.

---

17. How do you debug or inspect VHAL on a running device/emulator?

A few practical tools, roughly in order of how often you'll reach for them:

- **`adb shell dumpsys car_service`** — Dumps CarService's internal state: every property it knows about, current cached values, registered listeners/subscriptions, and configuration details. This is usually your first stop when something "isn't updating" — you can see whether CarService even *has* a current value for the property you care about, and whether your app's callback is registered.
- **`adb logcat`**, filtered on relevant tags — `VehicleHal`, `CarServiceHelper`, `CarPropertyService`, or the HAL service's own tag (often something like `android.hardware.automotive.vehicle`). You'll see logs for property reads/writes, subscription requests, and any errors VHAL reports back via `onPropertySetError`.
- **Process inspection** — `adb shell ps -A | grep vehicle` to confirm the VHAL process is actually running, and check its PID hasn't restarted unexpectedly (a restarting VHAL process often indicates crashes).
- **VHAL emulator script** (for the reference/emulator implementation) — typically run on your host machine, communicating over an `adb forward`'ed port to the VHAL process on the device/emulator. Lets you send arbitrary property-value injections — e.g., simulate `NIGHT_MODE = true` to test your app's dark theme switching, or set `FUEL_LEVEL` to a low value to test a low-fuel warning UI, all without needing real sensor input.
- **Service registry inspection** — for HIDL-era HALs, `lshal` lists registered HAL services; for AIDL services, you can inspect the service manager (`adb shell dumpsys -l | grep -i vehicle`) to confirm the `IVehicle` AIDL service is registered and discoverable.

A common debugging pattern: if an app's UI isn't updating, check (1) is the app's callback registered (`dumpsys car_service`), (2) is VHAL actually sending `onPropertyEvent`s

(`logcat`), and (3) does the property's `changeMode`/`access` even support what you're trying to do (re-check `VehiclePropConfig`).

---

18. What's the overall flow when an app subscribes to a continuously-changing property like vehicle speed?

Putting together everything above into one end-to-end sequence:

1. **App registers a callback**: `carPropertyManager.registerCallback(callback, PERF_VEHICLE_SPEED, SENSOR_RATE_NORMAL)`. This is a Binder call from the app process into CarService.
2. **CarService checks permission**: confirms the app holds `android.car.permission.CAR_SPEED`. If not, throws immediately.
3. **CarService manages the subscription**: it checks whether *any* app is already subscribed to `PERF_VEHICLE_SPEED`. If this is the first subscriber, CarService calls VHAL's `subscribe(callback, [SubscribeOptions{prop: PERF_VEHICLE_SPEED, sampleRate: 10Hz}])` — where `callback` here is CarService's own `IVehicleCallback` implementation, and the sample rate is chosen based on `SENSOR_RATE_NORMAL` (and clamped to the property's `minSampleRate`/`maxSampleRate` from its `VehiclePropConfig`). If a second app subscribes at a *higher* rate later, CarService may re-subscribe to VHAL at the new higher rate (since VHAL only knows about one subscription per property from CarService's perspective).
4. **VHAL starts producing data**: depending on the implementation, this might mean polling a real speed sensor/CAN signal at 10Hz, or (for the reference implementation) running an internal timer that updates a simulated value.
5. **VHAL pushes updates**: for each sample, VHAL calls `callback.onPropertyEvent([VehiclePropValue{prop: PERF_VEHICLE_SPEED, areaId: 0, value: {floatValues: [currentSpeed]}, timestamp: ..., status: AVAILABLE}])`.
6. **CarService receives the event**: its `IVehicleCallback.onPropertyEvent` implementation fires, it looks up which app-level listeners are registered for `PERF_VEHICLE_SPEED`, wraps the value in a `CarPropertyValue`, and dispatches it.
7. **App receives the update**: the app's `CarPropertyEventCallback.onChangeEvent(CarPropertyValue)` fires (on a Binder callback thread — apps typically need to post UI updates to the main thread from here), and the UI updates with the new speed.
8. **Unsubscribing**: when the app calls `unregisterCallback`, CarService removes that app's listener; if it was the *last* listener for that property, CarService calls VHAL's `unsubscribe()` so VHAL can stop producing/sending updates it no longer needs to.

This flow illustrates the recurring theme across all of VHAL's design: **CarService is the multiplexer and policy-enforcer, VHAL is the single source of truth for vehicle state, and the AIDL `IVehicle`/`IVehicleCallback` pair is the narrow, well-defined contract**

connecting them — regardless of what's actually happening underneath VHAL on any given vehicle.